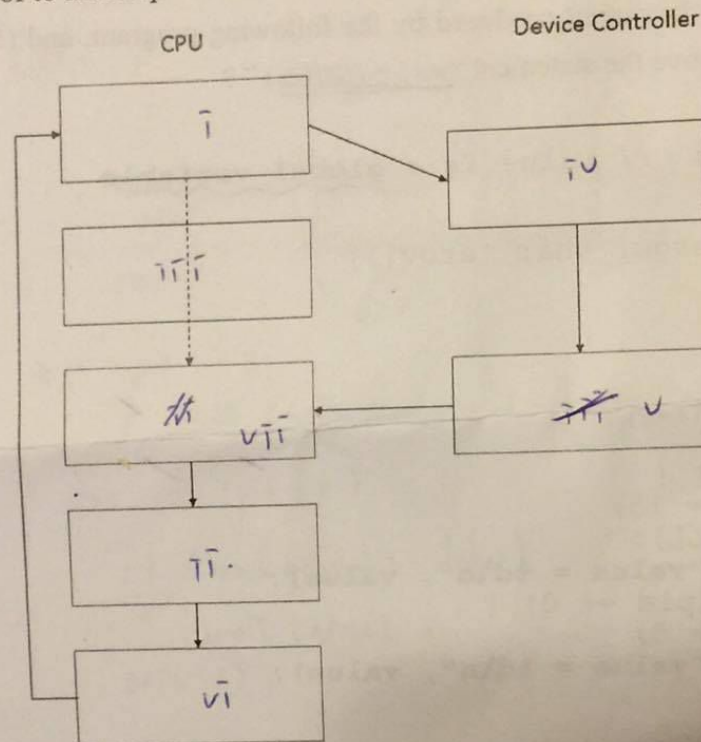


1. Interrupt, I/O operation, and multi-programming

(a) (5 points) How does the modern interrupt mechanism accelerate an interrupt operation? 加进 interrupt → table

(b) (10 points) The interrupt-driven I/O cycle consists of (i) Input ready, output complete, or error generates interrupt signal, (ii) Interrupt handler processes data, return from interrupt, (iii) CPU executing checks for interrupts between instructions, (iv) device driver initiates I/O, (v) Controller initiates I/O, (vi) CPU resumes processing of interrupted process (task), and (vii) CPU receiving interrupt, transfer control to interrupt handler. Organize the steps with the following flowchart.



(c) (10 points) What is the benefit of multi-programming? Explain the difference between it and time-sharing. → multiprogramming a extension

(d) (5 points) How do operating systems support new devices? → device driver

2. System calls, operating system design, and implementation: 關係講清楚

(a) (10 points) Describe how a system call is invoked using an API function? You **need to mention** software interrupt, dual-mode, system call number (index), and table of code pointer.

(b) (5 points) Explain why programmers prefer programming according to API functions rather than invoking system calls directly.

(c) (5 points) Describe the pros and cons of layered operating system structure.

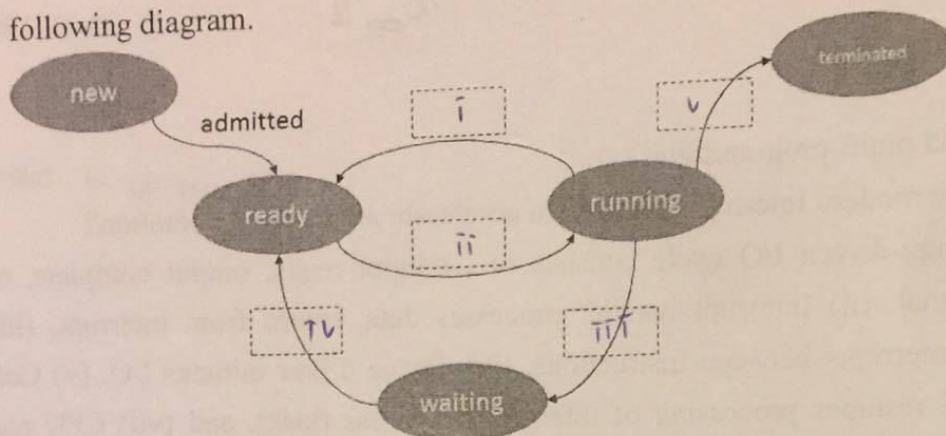
好处

一层一层

优缺点

3. Process state:

- (a) (10 points) During its life cycle, a process can (i) be interrupted, (ii) be dispatched by a short-term scheduler, (iii) perform I/O, (iv) have I/O completed, (v) exit. Organize these operations with the following diagram.



- (b) (5 points) Show the output produced by the following program, and (5 points) explain what would happen if we remove the statement "wait(NULL)"?

int value = 5; // value is a global variable

```

int main(int argc, char *argv[])
{
    pid_t pid;
    pid = fork();

    if(pid > 0) {
        value += 15;
        wait(NULL);
        printf("value = %d\n", value);
    } else if (pid == 0) {
        value += 5;
        printf("value = %d\n", value);
    }
}
  
```

4. Multithreaded Programming:

为什么 thread 比较便宜。

- (a) (5 points) Explain why allocating memory for process creation is costly as compared with thread creation.

- (b) (10 points) Describe the pros and cons of the one-to-one thread model.

5. (15 points) Complete the following C++ program which serves as a simple shell interface. The shell accepts user commands and then executes each command in a separate process. The shell allows the child process to run in the background (or concurrently) if the command ends with `&`. For simplicity, we assume that users never enter invalid commands and no arguments follow a command.

int main(void)

{
char inputBuffer[80]; // buffer to hold command entered
bool background; // true if a command is followed by '&'

child

while (CM)

{
cout << endl << " COMMAND->" ;
pid_t pid;

// step 1
// accept user command and check if the command is
// followed by '&'

CM = inputBuffer ;
↗

// step 2
// fork a child process, if background == false,
// the parent will wait
// the child process invoke exec(inputBuffer) to
// execute user command

pid = fork();

} // end of while

end of main

if (pid == 0)

exec (inputBuffer) ;

else if (pid > 0)

if (background == false)

wait (NULL) ;

exit (0) ;

一直接受指令

child 在背景執行

concurrently

双亲 - 操作